

Big Data

Curs 1

Big Data reprezintă un ansamblu de date generate și colectate continuu în mediul digital actual. Aceste date provin din utilizarea sistemelor informatice, a dispozitivelor mobile, a aplicațiilor online, a infrastructurilor industriale și a senzorilor integrați în diverse obiecte. Cantitatea de date este măsurată frecvent în zettabytes și crește pe măsură ce activitățile umane și tehnice sunt tot mai des monitorizate digital.

Firmele utilizează **Big Data** pentru a susține procesele decizionale, pentru a analiza funcționarea sistemelor interne și pentru a adapta produse și servicii în funcție de comportamentul utilizatorilor. Big Data **NU** se referă doar la cantitatea de date, ci și la modul în care acestea sunt colectate, stocate, prelucrate și analizate.

Big Data este definit printr-un set de caracteristici cunoscute sub denumirea de „V-uri”. Inițial, literatura de specialitate a identificat 3V, iar ulterior modelul a fost extins la 5V, 7V sau chiar 9V, pentru a reflecta mai bine complexitatea fenomenului.

Cele 7V ale Big Data

1. **Volume** (volum) se referă la cantitatea de date generate și stocate. Big Data presupune volume de date care depășesc capacitatea sistemelor tradiționale de baze de date. Aceste volume necesită soluții distribuite de stocare și procesare, precum sisteme de fișiere distribuite și platforme de calcul paralel.

2. **Velocity** (viteză) descrie ritmul în care datele sunt generate, transmise și procesate. Uneori datele sunt produse în timp real sau aproape în timp real, cum ar fi fluxurile de date din senzori, tranzacțiile online sau evenimentele din aplicații. Pentru această caracteristică se impune mecanisme de procesare rapidă și sisteme capabile să reacționeze într-un timp scurt.

Exemple → mecanisme de procesare rapidă și sisteme capabile să reacționeze într-un timp scurt”

- Analiza activității online → platformele de comerț electronic analizează comportamentul utilizatorilor în timp real, sistemul recomandă produse în timpul sesiunii utilizatorului.
- Detectarea fraudei bancare
- Monitorizarea traficului rutier
- Senzorii și camerele de trafic generează date în mod continuu despre fluxul de vehicule, datele sunt colectate în timp real, sunt analizate imediat pentru a identifica ambuteiaje sau accidente, semafoarele inteligente își pot modifica timpii de funcționare.
- Monitorizarea echipamentelor industriale → senzorii măsoară temperatura, vibrațiile și presiunea utilajelor, datele sunt transmise continuu, algoritmi detectează valori anormale, sistemul poate opri automat echipamentul pentru a preveni defectarea.

Se impune existența mecanismelor de procesare rapidă și sisteme capabile să reacționeze rapid, deoarece datele sunt generate continuu, iar valoarea lor depinde de capacitatea sistemului de a le analiza imediat,

3. **Variety** (varietate) - diversitatea tipurilor de date. Big Data include date structurate, semi-structurate și nestructurate, cum ar fi: loguri de sistem, imagini, fișiere audio, date etc. Gestionarea acestei varietăți necesită tehnologii flexibile.

4. **Veracity** (veridicitate) -calitatea și fiabilitatea datelor. Datele pot conține erori, valori lipsă sau informații incomplete. Analiza Big Data trebuie să ia în considerare aceste aspecte pentru a evita concluzii incorecte și pentru a gestiona incertitudinea asociată datelor.

5. **Value** (valoare) - reprezintă utilitatea practică a datelor. Nu toate datele colectate generează automat beneficii. Valoarea apare atunci când datele sunt analizate și transformate în informații relevante pentru luarea deciziilor sau pentru optimizarea proceselor.

6. **Variability** (variabilitate) - schimbarea semnificației sau structurii datelor în timp. Exemplu → același tip de date poate avea interpretări diferite în funcție de context sau perioadă.

7. **Visualization** (vizualizare) - modalitatea prin care rezultatele analizelor sunt prezentate. Reprezentările grafice sunt necesare pentru a facilita înțelegerea informațiilor, pentru a ajuta procesul decizional.

Extinderea la modelul celor **9V** - unele abordări includ încă două caracteristici suplimentare:

8. **Validity** (validitate) - măsura în care datele sunt corecte și potrivite scopului pentru care sunt utilizate. Chiar dacă datele sunt corecte din punct de vedere tehnic, ele pot fi neadecvate pentru anumite analize sau decizii.

9. **Volatility** (volatilitate) - durata de viață a datelor și perioada în care acestea își păstrează relevanța. Unele date sunt utile doar pe termen scurt (de exemplu, datele de trafic în timp real), în timp ce altele pot fi arhivate și analizate pe termen lung.

Date și Big Data

Datele structurate sunt cele mai ușor de organizat și de analizat deoarece respectă un format fix, bine definit. Acestea sunt aranjate în tabele cu rânduri și coloane, unde fiecare câmp are un tip de date clar stabilit. **Datele structurate** pot fi gestionate eficient cu baze de date relaționale; folosesc **SQL (Structured Query Language)** pentru interogare și analiză; sunt ușor de căutat, filtrat și agregat; permit analize rapide și precise. Chiar dacă datele structurate pot exista în volume foarte mari, **ele nu sunt întotdeauna considerate Big Data**, deoarece sunt relativ ușor de administrat cu tehnologiile clasice. Totuși, atunci când volumul, viteza sau varietatea cresc semnificativ, pot deveni parte a sistemului Big Data.

Date nestructurate - nu au un format predefinit și nu pot fi stocate ușor în tabele relaționale. Ele reprezintă cea mai mare parte a datelor generate în prezent și sunt mult mai dificil de analizat. **Exemple:** postări și comentarii pe rețelele sociale; fișiere audio și video; imagini și fotografii; recenzii și feedback de la clienți; documente text libere (PDF, Word). **Date nestructurate** nu au o schemă fixă; necesită tehnologii avansate pentru analiză; sunt dificil de indexat și căutat; analiza lor tradițională era costisitoare și lentă.

Pentru gestionarea datelor nestructurate se folosesc:

- lacuri de date (data lakes);
- baze de date NoSQL;
- tehnologii de inteligență artificială și machine learning;
- procesare distribuită (ex: Hadoop, Spark).

Analiza datelor nestructurate oferă informații valoroase despre comportamentul utilizatorilor, opinii, emoții și tendințe.

Date semi-structurate - reprezintă o combinație între date structurate și nestructurate. Ele nu respectă un model rigid, dar conțin metadata sau etichete care le oferă un anumit nivel de organizare.

Exemple: e-mailuri, fișiere XML sau JSON; date generate de senzori și dispozitive IoT; etichete de timp și locație (GPS).

Date **semi-structurate** conțin atât informații organizate, cât și neorganizate; sunt mai flexibile decât datele structurate; pot fi analizate mai ușor decât datele complet nestructurate; sunt foarte comune în aplicațiile moderne.

Observație: bazele de date moderne, împreună cu inteligența artificială, pot: identifica automat tipul de date; extrage structura din conținutul liber; crea algoritmi în timp real pentru analiză eficientă. Clasificarea datelor în structurate, nestructurate și semi-structurate este esențială pentru alegerea tehnologiilor potrivite de stocare și analiză.

- Datele structurate sunt ușor de gestionat și analizat.
- Datele nestructurate sunt cele mai complexe, dar și cele mai valoroase.
- Datele semi-structurate oferă flexibilitate și sunt din ce în ce mai utilizate.

Pentru Big Data, capacitatea de a integra și analiza toate aceste tipuri de date reprezintă un avantaj strategic major pentru organizații.

Importanța analizelor Big Data și rolul Apache Spark

Analiza Big Data permite extragerea de informații relevante din volume mari de date colectate din surse diverse. Aceste analize se bazează pe tehnici precum **analiza statistică, învățarea automată și inteligența artificială** și sprijină procesele de planificare, monitorizare și evaluare în domenii variate ca: economie, industrie, sănătate etc.

Volumele mari de date nu pot fi analizate folosind sisteme tradiționale, deoarece acestea nu sunt concepute pentru procesare **distribuită și paralelă**. **Apache Spark** joacă un rol central în ecosistemul Big Data.

Apache Spark este un motor de procesare distribuită care permite analiza rapidă a datelor pe **clustere de calcul**. El oferă suport pentru:

- procesare batch a seturilor mari de date,
- procesare de fluxuri de date (streaming),
- analiză statistică,
- învățare automată,
- interogare **SQL asupra datelor distribuite**.

Astfel, Spark face posibilă transformarea datelor brute în informații utilizabile, reducând timpul necesar analizelor și permițând rularea de algoritmi complecși pe volume mari de date.

Big Data este strâns legat de **dezvoltarea tehnologiilor de procesare distribuită, deoarece datele sunt:**

- prea numeroase pentru a fi stocate și procesate pe un singur sistem;
- generate continuu;
- provenite din surse eterogene;

Apache Spark abordează aceste provocări prin: distribuirea datelor pe mai **multe noduri**, **executarea paralelă a operațiilor**, utilizarea memoriei pentru a reduce timpii de procesare, suportul pentru mai multe limbaje (**Python, SQL, Scala, R**).

Spark reprezintă o componentă esențială în analiza Big Data, permițând aplicarea tehnicilor analitice avansate asupra unor volume de date care altfel ar fi dificil de gestionat.

Creșterea Big Data este asociată cu evoluția tehnologică. Capacitatea de stocare și de procesare a crescut constant, iar numărul dispozitivelor care generează date crește continuu și seturile de date se modifică rapid și necesită analiză frecventă.

Soluțiile Big Data trebuie **să fie scalabile**, adică să permită adăugarea de resurse de calcul pe măsură ce volumul de date crește.

Apache Spark este conceput pentru a funcționa în astfel de medii scalabile. El poate rula pe clustere mici sau mari și poate fi integrat cu sisteme de stocare distribuite, cum ar fi HDFS (<https://www.ibm.com/think/topics/hdfs>) sau Delta Lake. Această flexibilitate explică utilizarea sa frecventă în platforme moderne de analiză Big Data, inclusiv Databricks.

Analizele Big Data sunt esențiale pentru valorificarea datelor colectate, iar Apache Spark oferă infrastructura necesară pentru realizarea acestor analize într-un mod distribuit și eficient. Legătura dintre **Big Data și Spark** este una directă: pe măsură ce volumul și complexitatea datelor cresc, devine necesară utilizarea **unor motoare de procesare** precum Spark, capabile să susțină analiza avansată la scară largă.

Sisteme de stocare distribuite în Big Data: HDFS și Delta Lake

Volumele mari de date nu pot fi stocate eficient pe un singur sistem în contextul Big Data. Astfel sunt utilizate sisteme de stocare distribuite, care împart datele pe mai multe noduri și permit accesul paralel la acestea. Două astfel de soluții frecvent întâlnite sunt **HDFS și Delta Lake**.

HDFS (Hadoop Distributed File System)

HDFS este un sistem de fișiere distribuit, dezvoltat ca parte a ecosistemului Apache Hadoop. Scopul său principal este stocarea fișierelor de mari dimensiuni pe un cluster de calcul format din mai multe noduri.

HDFS este conceput pentru:

- stocarea unor volume mari de date,
- toleranță la erori,
- acces secvențial eficient la date.

Cum funcționează HDFS?

HDFS împarte fișierele mari în blocuri (128 MB sau 256 MB), care sunt distribuite pe mai multe noduri din cluster. Arhitectura HDFS include:

- NameNode – gestionează metadatele (structura fișierelor, locația blocurilor),
- DataNode – stochează efectiv blocurile de date.

Pentru a asigura fiabilitatea, fiecare bloc este replicat pe mai multe noduri. Astfel, dacă un nod cade, datele pot fi accesate dintr-o altă replică.

HDFS este potrivit pentru: stocarea datelor brute (raw data), procesare batch, aplicații în care datele sunt scrise o singură dată și citite de mai multe ori.

Apache Spark poate citi și scrie date din HDFS, folosindu-l ca strat de stocare distribuit.

Limitările HDFS

Deși este eficient pentru stocare, HDFS: nu oferă tranzacții, nu asigură consistență la nivel de tabel, nu gestionează modificări frecvente (update, delete) eficient, nu oferă metadate avansate pentru analiză.

Aceste limitări au condus la apariția unor soluții moderne, ca Delta Lake.

Delta Lake - este un strat de stocare deschis, construit peste sisteme de fișiere distribuite (precum HDFS sau stocarea cloud), care adaugă funcționalități de tip bază de date peste fișierele de date.

În Databricks, Delta Lake este utilizat ca format principal pentru stocarea datelor.

Delta Lake extinde stocarea clasică prin:

- tranzacții ACID (Atomicitate, Consistență, Izolare, Durabilitate),
- versionarea datelor (time travel),
- gestionarea modificărilor (UPDATE, DELETE, MERGE),
- schema enforcement și schema evolution,
- metadate consistente pentru analize.

Datele sunt stocate sub formă de fișiere Parquet, iar un transaction log menține evidența modificărilor.

Delta Lake este potrivit pentru:

- pipeline-uri de date (ETL),
- procesare batch și streaming unificate,
- analiză Big Data cu Apache Spark,
- aplicații care necesită consistență și fiabilitate.

Apache Spark se integrează nativ cu Delta Lake, permițând citirea și scrierea datelor într-un mod tranzacțional.

În Databricks:

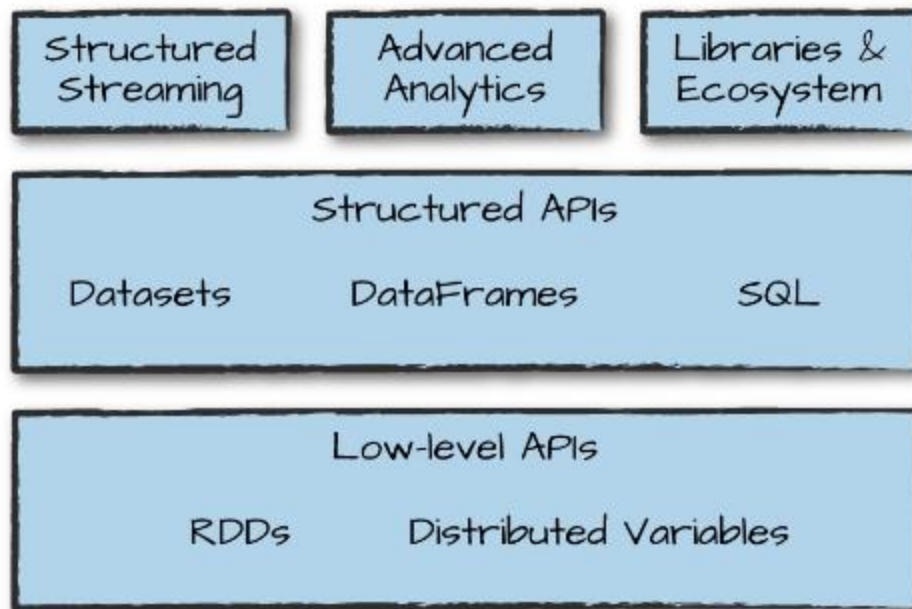
- stocarea este de regulă realizată pe cloud (S3, ADLS, GCS),
- Delta Lake este utilizat ca strat logic peste această stocare,
- Apache Spark este motorul de procesare.

Arhitectura tipică este: **Stocare distribuită → Delta Lake → Apache Spark → Analiză Big Data**

HDFS și Delta Lake joacă roluri diferite în conceptul Big Data. HDFS oferă o soluție de stocare distribuită pentru volume mari de date, în timp ce Delta Lake adaugă mecanisme necesare pentru consistență, versionare și gestionarea modificărilor. Apache Spark se integrează cu ambele, însă în platforme moderne precum Databricks, Delta Lake este preferat pentru analiza Big Data și pentru pipeline-uri de date fiabile.

Ce este Apache Spark?

Apache Spark este un motor de calcul unificat și un set de biblioteci pentru procesarea paralelă a datelor pe clustere de computere. Spark este cel mai activ dezvoltat motor open-source pentru această sarcină, devenind un standard pentru orice dezvoltator sau specialist interesat de big data. Spark acceptă mai multe limbaje de programare utilizate pe scară largă (Python, Java, Scala și R), include biblioteci pentru sarcini diverse, de la SQL la streaming și machine learning, și poate rula oriunde, de la un laptop la clustere de mii de servere. Acest lucru îl face un sistem ușor de început și scalabil pentru procesarea datelor mari sau foarte mari.



https://analyticsdata24.wordpress.com/wp-content/uploads/2020/02/spark-the-definitive-guide40www.bigdatabugs.com_.pdf

Figura 1 - ilustrează toate componentele și bibliotecile pe care Spark le oferă utilizatorilor finali.

Figura este împărțită pe **niveluri**, fiecare reprezentând o categorie diferită de funcționalități oferite de Spark.

1. **Nivelul bibliotecilor de top (High-level Libraries)** - aceste componente sunt plasate în partea superioară pentru că reprezintă **funcționalități avansate**, construite peste API-urile de bază.

Componenta Structured Streaming: permite procesarea **datelor în timp real**.

Funcționează cu aceleași API-uri ca și procesarea batch. Oferă un mod simplu, declarativ, de a defini fluxuri continue de date (stream processing).

Componenta Advanced Analytics este utilizată pentru modele statistice, clasificare, regresie, clustering și conține componentele: **MLI** → machine learning și **GraphX** → analiza grafurilor

Componenta Libraries & Ecosystem extinde Spark spre un ecosistem complet de analiză și procesare; reunește toate bibliotecile și extensiile oficiale Spark și din comunitate, care permit integrarea cu:

- Hadoop
- Kafka
- Delta Lake
- Instrumente pentru ingestia și stocarea datelor

2. Nivelul API-urilor structurate (Structured APIs)

Datasets oferă siguranță de tip (type safety) + optimizări și conține structuri de date tipizate (typed) folosite mai ales în Scala și Java.

DataFrames este cel mai utilizat API pentru procesare la scară mare și conține tabele distribuite de date (similar cu tabelele SQL); include optimizări prin **Catalyst** (motorul de optimizare al Spark).

SQL permite scrierea interogărilor în SQL pur.

3. Nivelul API-urilor de bază (Low-level APIs) sunt API-urile originale ale Spark (Spark 1.x), care oferă control complet asupra modului în care sunt distribuite datele.

RDDs (Resilient Distributed Datasets)

- Structura fundamentală a Spark.
- Permite control manual asupra distribuției, particionării și transformărilor.
- Este un API mai detaliat, dar mai greu de optimizat automat.

Distributed Variables permite mecanisme pentru partajarea datelor între noduri:

- **Broadcast variables** → distribuirea unor valori mari către toate nodurile
- **Accumulators** → agregări globale

Observație:

- **Low-level APIs (RDDs + Distributed Variables)** sunt fundația Spark,
- **Structured APIs (DataFrames, Datasets, SQL)** se bazează pe RDD-uri,
- **High-level Libraries** (Streaming, MLlib etc.) sunt construite peste Structured APIs, facilitând dezvoltarea aplicațiilor complexe.

Obiectiv: Apache Spark ofera o **platformă unificată** pentru dezvoltarea aplicațiilor de tip big data. Nevoia unui astfel de sistem unificat a aparut deoarece Spark este proiectat să susțină o gamă vastă de sarcini de analiză a datelor - de la încărcarea și procesarea simplă a datelor, până la interogări SQL, machine learning și procesare de streaming. Toate aceste sarcini folosesc același motor de calcul și un set coerent de API-uri, ceea ce permite dezvoltarea unor aplicații complexe fără a combina tehnologii disparate.

Acest obiectiv este inspirat de realitatea sistemelor moderne de analiză a datelor, care trebuie să îmbine multe instrumente diferite ce de exemplu aplicații interactive precum notebook-urile Jupyter, procese tradiționale de producție, etc. Spark simplifică această complexitate oferind o arhitectură unitară.

Natura unificată a Spark permite construirea aplicațiilor ușor și eficient. În primul rând, Spark oferă API-uri compozabile, consecvente, care le permit dezvoltatorilor să creeze aplicații din părți mai mici, integrate cu bibliotecile existente. În al doilea rând, Spark include optimizări interne care îmbunătățesc performanța prin combinarea și optimizarea operațiunilor sub forma unui plan comun.

De exemplu, dacă o aplicație încarcă datele folosind SQL și apoi aplică un model de machine learning, motorul Spark poate combina și optimiza aceste etape într-o singură executare eficientă.

Această combinație dintre API-uri generale și execuție de înaltă performanță face ca Spark să fie o platformă puternică atât pentru aplicații interactive, cât și pentru aplicații de producție.

Focalizarea Spark pe ideea unei platforme unificate nu este unică - multe alte domenii au beneficiat de astfel de platforme comune. De exemplu, oamenii de știință din domeniul datelor folosesc adesea platforme comune în Python sau R pentru modelare, iar dezvoltatorii web folosesc framework-uri unificate precum Node.js sau Django. Totuși, înainte de Spark, nu exista un sistem open-source care să ofere un astfel de cadru unificat pentru prelucrarea paralelă a datelor. Utilizatorii trebuiau să îmbine manual diverse instrumente și API-uri.

Spark a schimbat rapid acest lucru, devenind standardul industriei pentru acest tip de procesare. De-a lungul timpului, Spark și-a extins colecția de API-uri pentru a acoperi tot mai multe tipuri de sarcini, păstrând totodată tema unui sistem unificat. Un aspect major

al acestei evoluții îl reprezintă **API-urile structurate** (DataFrames, Datasets și SQL), introduse în Spark 2.0, care permit optimizări suplimentare și aplicații mult mai eficiente.

Istoria Spark

În trecut, computerele deveneau automat mai rapide în fiecare an datorită creșterii vitezei procesoarelor, iar aplicațiile funcționau tot mai bine fără să fie nevoie de modificări. Această evoluție a permis dezvoltarea unui ecosistem uriaș de programe construite pentru a rula pe un singur procesor. Totul s-a schimbat însă în jurul anului 2005, când producătorii de hardware nu au mai reușit să crească frecvența procesoarelor din cauza limitelor fizice, orientându-se în schimb către procesoare cu mai multe nuclee, care necesită paralelism pentru a obține performanță. În același timp, costurile de stocare și colectare a datelor au scăzut dramatic, iar organizațiile au început să acumuleze volume uriașe de informații provenite din senzori, camere, sisteme industriale sau date publice. **Astfel a apărut o lume în care este extrem de ieftin să colectezi date, dar foarte costisitor și dificil să le procesezi cu aplicațiile tradiționale, care nu pot profita automat de paralelism. Acest dezechilibru a creat nevoia unor modele de programare noi, capabile să ruleze calcule distribuite pe clustere de mașini.** Apache Spark a fost conceput exact pentru acest context modern, oferind un motor de procesare paralelă eficient, adaptat volumelor mari de date și bazat pe principiile necesare pentru a lucra în această eră a big data.

Apache Spark a început **la UC Berkeley în 2009** ca proiectul de cercetare Spark, publicat pentru prima dată în anul următor într-o lucrare intitulată „*Spark: Cluster Computing with Working Sets*”, realizată de Matei Zaharia, Mosharaf Chowdhury, Michael Franklin, Scott Shenker și Ion Stoica din cadrul AMPLab, UC Berkeley. La vremea aceea, **Hadoop MapReduce** era motorul dominant pentru programare paralelă pe clustere, fiind primul sistem open-source care aborda procesarea paralelă a datelor pe clustere de mii de noduri. AMPLab colaborase cu numeroși utilizatori timpurii ai MapReduce pentru a înțelege avantajele și limitările acestui model de programare și astfel a reușit să identifice o listă de probleme comune și să înceapă proiectarea unor platforme de calcul mai generale.

În plus, Zaharia lucrase cu **utilizatori Hadoop** de la UC Berkeley pentru a înțelege nevoile acestora - în mod special echipe care realizau machine learning la scară largă folosind algoritmi iterativi ce necesitau mai multe treceri peste aceleași date.

Din aceste discuții au reieșit două concluzii importante. În primul rând, calculul pe clustere avea un potențial enorm: în fiecare organizație care folosea MapReduce, apăreau aplicații noi construite pe baza datelor existente, iar tot mai multe grupuri adoptau această tehnologie. În al doilea rând, motorul MapReduce făcea dificilă și ineficientă dezvoltarea aplicațiilor complexe. De exemplu, un algoritm tipic de machine learning poate necesita 10–20 de treceri peste date, iar în MapReduce fiecare trecere trebuia implementată ca un job separat, lansat individual pe cluster și care reîncărca datele de la zero.

Pentru a rezolva această problemă, echipa Spark a proiectat mai întâi un **API bazat pe programare funcțională, capabil să exprime concis aplicații cu mai multe etape**. Apoi, ei au implementat acest API peste un motor nou, capabil să partajeze eficient date în memorie între pașii de calcul. Sistemul a fost testat atât cu utilizatori din Berkeley, cât și din afara universității.

Prima versiune Spark suporta doar aplicații batch, dar curând a apărut un alt caz de utilizare important: știința datelor interactivă și interogările ad hoc. Prin conectarea interpretului Scala la Spark, proiectul putea oferi un sistem interactiv foarte ușor de utilizat pentru a rula interogări pe sute de mașini. AMPLab a dezvoltat apoi Shark, un motor capabil să ruleze

interogări SQL peste Spark și să permită folosirea sa de către analiști și oameni de știință ai datelor. Spark a fost lansat prima dată în 2011.

După aceste lansări inițiale, a devenit clar că cele mai puternice completări pentru Spark ar fi bibliotecile noi. Proiectul a început astfel să urmeze abordarea „bibliotecilor standard”, pe care o păstrează și astăzi. Anumite grupuri din AMPLab au creat MLlib, Spark Streaming și GraphX. Ei s-au asigurat că aceste API-uri sunt interoperabile, permițând dezvoltarea aplicațiilor big data complete în același motor pentru prima dată.

În 2013, proiectul crescuse masiv, având peste 100 de contribuitori din peste 30 de organizații din afara UC Berkeley. AMPLab a donat Spark către Apache Software Foundation, oferind proiectului un mediu stabil pe termen lung, independent de furnizori. Tot atunci, echipa inițială AMPLab a lansat compania **Databricks** pentru a consolida proiectul, alăturându-se comunității mai largi de companii ce contribuiau la Spark. De atunci, comunitatea Apache Spark a lansat Spark 1.0 în 2014 și Spark 2.0 în 2016 și continuă să lanseze versiuni regulate cu noi funcționalități.

În cele din urmă, conceptul de bază al Spark - API-uri compozabile - a evoluat în timp. Versiunile timpurii (înainte de 1.0) defineau API-ul în termeni de operații funcționale - transformări paralele precum map și reduce pe colecții de obiecte Java. Odată cu Spark 1.0, proiectul a adăugat Spark SQL, un API nou pentru lucrul cu date structurate - tabele cu format fix, independent de reprezentarea lor în memoria Java.

Spark SQL a permis optimizări mult mai puternice prin înțelegerea atât a formatului datelor, cât și a codului utilizatorului. Ulterior, proiectul a adăugat o multitudine de API-uri noi construite pe această fundație, precum DataFrames, pipeline-uri de machine learning și Structured Streaming, un API avansat și optimizat pentru streaming.

Cum funcționează Big Data

Big Data funcționează prin transformarea unor volume foarte mari de date brute în informații relevante și utile pentru luarea deciziilor. Acest proces presupune colectarea, stocarea, procesarea și analiza datelor cu ajutorul tehnologiilor moderne, precum cloud computing, baze de date distribuite și inteligență artificială.

Big data processing and the iterative data pipeline



Fig 2. Etapele procesului Big Data

În figura 2 se observă că funcționarea Big Data este organizată sub forma unui pipeline de date iterativ, alcătuit din mai multe etape succesive, care permit gestionarea eficientă a datelor și obținerea de insight-uri valoroase.

1. **Sursele de date (Data sources)**

Datele sunt generate din surse variate, precum rețele sociale, aplicații, senzori sau sisteme informatice. Aceste date sunt eterogene și pot fi structurate, nestructurate sau semi-structurate.

2. **Colectarea datelor (Ingestion)**

În această etapă are loc preluarea volumelor mari de date din surse disparate. Colectarea se realizează în timp real sau în batch, utilizând tehnologii specializate, capabile să gestioneze fluxuri continue de date.

3. **Stocarea datelor (Storage)**

Datele colectate sunt stocate în sisteme scalabile, precum cloud-ul, data lakes sau sisteme distribuite, care permit extinderea rapidă și elimină limitările soluțiilor tradiționale.

4. **Procesarea datelor (Processing)**

Datele brute sunt curățate, transformate și organizate pentru a putea fi analizate eficient. Această etapă este esențială pentru asigurarea calității datelor.

5. **Analiza și vizualizarea datelor (Analysis & Visualization)**

Datele sunt analizate folosind algoritmi de inteligență artificială și machine learning, iar rezultatele sunt prezentate sub formă de grafice, rapoarte și dashboard-uri, ușor de interpretat de către factorii de decizie.

Procesul ilustrat în figura 2 este **iterativ**, ceea ce înseamnă că rezultatele analizei pot influența etapele anterioare. Modelele analitice se optimizează continuu, iar sistemul învață din datele procesate anterior, îmbunătățind constant acuratețea și relevanța informațiilor obținute.

Procesarea Big Data în arhitectura modernă (Modern Architecture Pipeline)

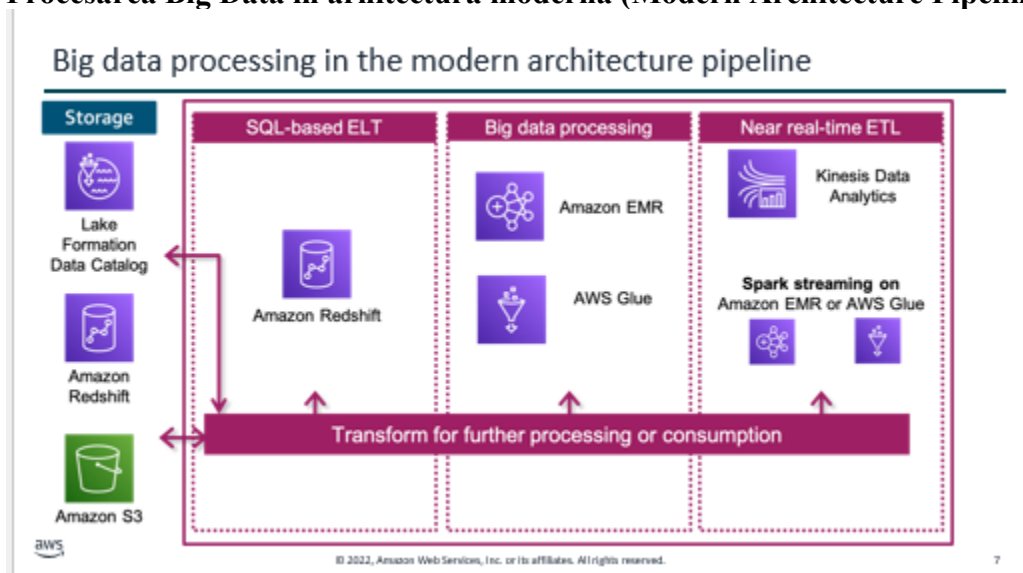


Fig 3. Modern Architecture Pipeline

Figura 3 ilustrează o **arhitectură modernă de procesare Big Data**, bazată pe tehnologii cloud, care permite colectarea, stocarea, procesarea și analiza datelor la scară largă, atât în mod batch, cât și în timp aproape real. Această arhitectură este un exemplu concret al modului în care funcționează Big Data în practică, folosind un pipeline de date flexibil și scalabil.

1. Stocarea datelor – Data Lake și catalogarea acestora

zona de stocare (Storage), reprezentată de un *data lake*. Aici sunt păstrate volume foarte mari de date brute, provenite din surse diverse.

- **Amazon S3** este utilizat ca spațiu principal de stocare pentru Big Data, datorită scalabilității ridicate.
- **AWS Lake Formation** și **Data Catalog** asigură organizarea, securizarea și descrierea datelor, facilitând accesul și guvernanta acestora.
- **Amazon Redshift** poate funcționa atât ca depozit analitic, cât și ca sursă pentru procesări ulterioare.

Această etapă corespunde fazei de **stocare Big Data**, unde datele nu sunt constrânse de limitele sistemelor tradiționale.

2. Procesarea SQL-based ELT

SQL-based ELT (Extract, Load, Transform).

- Datele sunt încărcate inițial în sistem (Load) și transformate ulterior.
- **Amazon Redshift** permite rularea de interogări SQL complexe direct pe volume mari de date.
- Acest model este eficient pentru date structurate și semi-structurate.

Această etapă demonstrează modul în care Big Data poate fi procesat folosind limbaje cunoscute (SQL), dar pe infrastructuri moderne, scalabile.

3. Procesarea Big Data (Batch Processing) este destinată seturilor mari de date și analizelor complexe.

- **Amazon EMR** este utilizat pentru procesarea distribuită a datelor (ex. Hadoop, Spark).

- **AWS Glue** este folosit pentru integrarea, curățarea și transformarea automată a datelor.

Această etapă corespunde fazei de **processing**, unde datele brute sunt pregătite pentru analiză, prin curățare, agregare și combinare.

4. Procesarea aproape în timp real (Near real-time ETL) - esențială pentru aplicațiile moderne Big Data.

- **Kinesis Data Analytics** permite analizarea fluxurilor de date în timp real.
- **Spark Streaming** pe Amazon EMR sau AWS Glue procesează datele pe măsură ce acestea sunt generate.

Această componentă răspunde cerinței de **viteză**, unul dintre cei cinci „V” ai Big Data, permițând obținerea rapidă de insight-uri acționabile.

5. Transformarea pentru procesare ulterioară sau consum - Transform for further processing or consumption, subliniază faptul că datele procesate:

- pot fi reutilizate în alte etape ale pipeline-ului;
- pot fi consumate de aplicații analitice, dashboard-uri sau modele AI;
- susțin caracterul **iterativ** al procesului Big Data.

Rezultatele obținute pot influența colectarea și procesarea viitoare a datelor, ducând la optimizarea continuă a sistemului.

Figura 3 evidențiază modul în care **Big Data este procesat într-o arhitectură modernă cloud**, printr-un pipeline flexibil care integrează:

- stocare scalabilă,
- procesare batch și în timp real,
- transformare și analiză avansată a datelor.

Această arhitectură confirmă faptul că Big Data nu este un proces liniar, ci **un ecosistem dinamic și iterativ**, care permite organizațiilor să transforme datele brute în informații valoroase și decizii strategice.

Tipuri de procesare a datelor (Types of Data Processing)

În cadrul arhitecturilor Big Data moderne, procesarea datelor se realizează în principal prin **două modele complementare: procesarea în batch și procesarea de tip streaming**. Alegerea uneia sau ambelor abordări depinde de **frecvența accesului la date, viteza necesară pentru analiză și tipul deciziilor care trebuie luate**.

1. Procesarea datelor în batch (Batch Data Processing)

Procesarea în batch presupune **prelucrarea unor volume mari de date la intervale de timp prestabilite**, nu în mod continuu. Datele sunt colectate pe o perioadă de timp, apoi procesate împreună ca un set unitar.

Caracteristici principale:

- datele sunt accesate **infrequent** (așa-numitele *cold data*);
- procesarea are loc la intervale regulate (zilnic, săptămânal etc.);
- suportă atât date structurate, cât și nestructurate;
- este potrivită pentru **analize complexe și aprofundate** asupra seturilor mari de date;
- nu necesită răspuns în timp real.

Tehnologii utilizate:

- **Amazon EMR**

- **Apache Hadoop**
- Apache Spark (mod batch)

Procesarea batch este eficientă din punct de vedere al costurilor și este esențială atunci când accentul cade pe **acuratețe și profunzimea analizei**, nu pe viteză.

2. Procesarea datelor de tip streaming (Streaming Data Processing)

Procesarea streaming presupune **analiza datelor pe măsură ce acestea sunt generate**, aproape în timp real. Datele sunt procesate incremental, fără a aștepta acumularea unor volume mari.

Caracteristici principale:

- datele sunt accesate **frecvent** (*hot data*);
- procesarea are loc continuu;
- este capabilă să gestioneze fluxuri mari de date imprevizibile;
- permite reacții rapide și decizii imediate;
- este esențială pentru aplicațiile sensibile la timp.

Exemple de utilizare:

- detectarea fraudelor în timp real;
- monitorizarea sistemelor și infrastructurilor;
- analiza comportamentului utilizatorilor online;
- aplicații IoT și senzori;
- recomandări personalizate în timp real.

Tehnologii utilizate:

- **Amazon Kinesis Data Streams**
- **Apache Spark Streaming**
- Apache Flink

Procesarea streaming răspunde cerinței de **viteză**, unul dintre cei cinci „V” ai Big Data, permițând obținerea de insight-uri imediat ce datele sunt generate.

Relația dintre batch și streaming

În arhitecturile moderne Big Data, **cele două tipuri de procesare nu se exclud**, ci sunt adesea utilizate împreună:

- batch → analiză istorică, aprofundată;
- streaming → analiză în timp real și reacții rapide.

Această combinație permite organizațiilor să obțină **o imagine completă** asupra datelor, atât din perspectiva trecutului, cât și a prezentului.

Framework-uri care suportă procesarea batch și streaming (Fig 4)

Frameworks that support batch and stream processing

Batch Processing	Stream Processing	
Apache Spark	Amazon Kinesis	
Apache Hadoop MapReduce	Apache Spark Streaming	AWS Lambda
Apache Hive	Apache Hive	Apache Flink
Apache Pig	Apache Pig	Apache Storm

Figura 4 prezintă **principalele framework-uri utilizate în ecosistemul Big Data** pentru procesarea datelor, grupate în funcție de tipul de procesare pe care îl susțin: **batch processing** și **stream processing**. Aceste framework-uri oferă infrastructura software necesară pentru gestionarea volumelor mari de date, atât în mod periodic, cât și în timp real.

1. Framework-uri pentru procesarea batch

Procesarea batch este orientată către **analiza volumelor mari de date istorice**, la intervale de timp prestabilite. Framework-urile din această categorie sunt optimizate pentru stabilitate, scalabilitate și analiză profundă.

Apache Spark

- Framework distribuit pentru procesarea rapidă a datelor.
- Suportă procesarea batch la scară mare.
- Este mai rapid decât Hadoop MapReduce datorită procesării in-memory.
- Utilizat frecvent pentru analize complexe și machine learning.

Apache Hadoop MapReduce

- Model clasic de procesare batch.
- Procesează datele în etape de tip *map* și *reduce*.
- Foarte robust, dar mai lent comparativ cu soluțiile moderne.
- Potrivit pentru volume extrem de mari de date.

Apache Hive

- Permite interogarea datelor mari folosind SQL (HiveQL).
- Rulează peste Hadoop.
- Ideal pentru data warehousing și analize istorice.

Apache Pig

- Limbaj de nivel înalt (Pig Latin) pentru procesarea batch.
- Simplifică scrierea fluxurilor de transformare a datelor.
- Utilizat mai ales pentru preprocesarea seturilor mari de date.

2. Framework-uri pentru procesarea streaming

Procesarea streaming este utilizată atunci când datele trebuie analizate **pe măsură ce sunt generate**, cu latență minimă. Framework-urile din această categorie sunt esențiale pentru aplicațiile în timp real.

Amazon Kinesis

- Serviciu AWS pentru procesarea fluxurilor de date în timp real.
- Scalabil și complet gestionat.
- Utilizat pentru loguri, evenimente, date IoT.

Apache Spark Streaming

- Extensie a Apache Spark pentru procesarea datelor în flux.
- Procesează datele în micro-batch-uri.
- Permite integrarea ușoară cu procesarea batch.

AWS Lambda

- Platformă serverless pentru rularea codului la evenimente.
- Foarte potrivită pentru procesarea în timp real.
- Elimină necesitatea administrării infrastructurii.

Apache Flink

- Framework avansat pentru procesare streaming nativă.
- Oferă latență foarte scăzută și procesare continuă.
- Suportă procesarea evenimentelor complexe.

Apache Storm

- Framework de procesare streaming în timp real.
- Conceput pentru latență foarte mică.
- Utilizat pentru aplicații critice, precum monitorizarea sistemelor.

3. Framework-uri hibride (batch + streaming)

Unele framework-uri, precum **Apache Spark**, **Apache Hive** și **Apache Pig**, apar în ambele categorii. Acest lucru subliniază faptul că **arhitecturile Big Data moderne sunt hibride**, permițând:

- analiză istorică (batch);
- analiză în timp real (streaming);
- reutilizarea acelorași instrumente și competențe.

Figura 4 evidențiază diversitatea framework-urilor disponibile pentru procesarea Big Data și arată că **nu există o soluție unică**, ci un ecosistem adaptabil diferitelor nevoi. Alegerea framework-ului potrivit depinde de:

- volumul datelor,
- viteza de procesare necesară,
- complexitatea analizelor,
- infrastructura disponibilă.

Firmele folosesc **combinații de framework-uri batch și streaming** pentru a obține insight-uri complete și în timp util.